

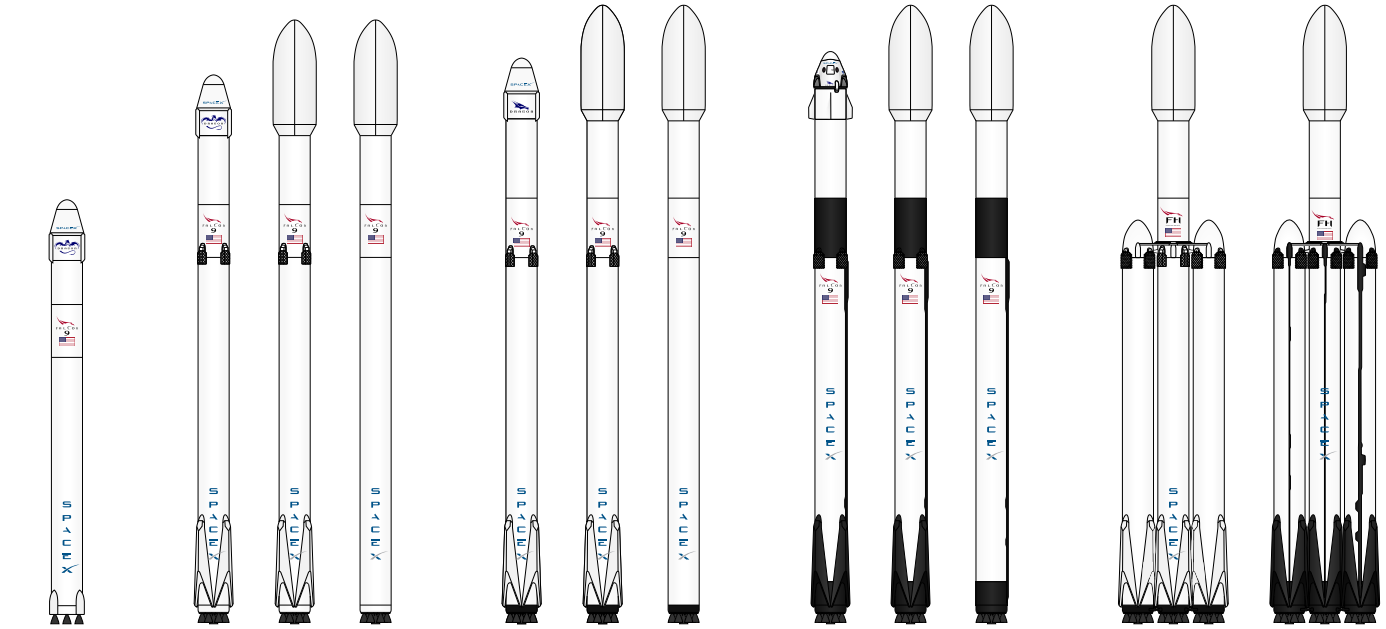
Space X Falcon 9 First Stage Landing Prediction

Web scraping Falcon 9 and Falcon Heavy Launches Records from Wikipedia

Estimated time needed: **40** minutes

In this lab, you will be performing web scraping to collect Falcon 9 historical launch records from a Wikipedia page titled `List of Falcon 9 and Falcon Heavy launches`

https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches



Falcon 9 first stage will land successfully



Several examples of an unsuccessful landing are shown here:



More specifically, the launch records are stored in a HTML table shown below:

2020 [edit]

In late 2019, Gwynne Shotwell stated that SpaceX hoped for as many as 24 launches for Starlink satellites in 2020,[490] in addition to 14 or 15 non-Starlink launches. At 26 launches, 13 of which for Starlink satellites, Falcon 9 had its most prolific year, and Falcon rockets were second most prolific rocket family of 2020, only behind China's Long March rocket family.[491]

[hide] Flight No.	Date and time (UTC)	Version, Booster[b]	Launch site	Payload[c]	Payload mass	Orbit	Customer	Launch outcome	Booster landing
78	7 January 2020, 02:19:21[492]	F9 B5 △ B1049.4	CCAFS, SLC-40	Starlink 2 v1.0 (60 satellites)	15,600 kg (34,400 lb)[5]	LEO	SpaceX	Success	Success (drone ship)
	Third large batch and second operational flight of Starlink constellation. One of the 60 satellites included a test coating to make the satellite less reflective, and thus less likely to interfere with ground-based astronomical observations.[493]								
79	19 January 2020, 15:30[494]	F9 B5 △ B1046.4	KSC, LC-39A	Crew Dragon in-flight abort test[495] (Dragon C205.1)	12,050 kg (26,570 lb)	Sub-orbital[496]	NASA (CTS)[497]	Success	No attempt
	An atmospheric test of the Dragon 2 abort system after Max Q. The capsule fired its SuperDraco engines, reached an apogee of 40 km (25 mi), deployed parachutes after reentry, and splashed down in the ocean 31 km (19 mi) downrange from the launch site. The test was previously slated to be accomplished with the Crew Dragon Demo-1 capsule;[498] but that test article exploded during a ground test of SuperDraco engines on 20 April 2019.[419] The abort test used the capsule originally intended for the first crewed flight.[499] As expected, the booster was destroyed by aerodynamic forces after the capsule aborted.[500] First flight of a Falcon 9 with only one functional stage — the second stage had a mass simulator in place of its engine.								
80	29 January 2020, 14:07[501]	F9 B5 △ B1051.3	CCAFS, SLC-40	Starlink 3 v1.0 (60 satellites)	15,600 kg (34,400 lb)[5]	LEO	SpaceX	Success	Success (drone ship)
	Third operational and fourth large batch of Starlink satellites, deployed in a circular 290 km (180 mi) orbit. One of the fairing halves was caught, while the other was fished out of the ocean.[502]								
81	17 February 2020, 15:05[503]	F9 B5 △ B1056.4	CCAFS, SLC-40	Starlink 4 v1.0 (60 satellites)	15,600 kg (34,400 lb)[5]	LEO	SpaceX	Success	Failure (drone ship)
	Fourth operational and fifth large batch of Starlink satellites. Used a new flight profile which deployed into a 212 km × 386 km (132 mi × 240 mi) elliptical orbit instead of launching into a circular orbit and firing the second stage engine twice. The first stage booster failed to land on the drone ship[504] due to incorrect wind data.[505] This was the first time a flight proven booster failed to land.								
82	7 March 2020, 04:50[506]	F9 B5 △ B1059.2	CCAFS, SLC-40	SpaceX CRS-20 (Dragon C112.3 △)	1,977 kg (4,359 lb)[507]	LEO (ISS)	NASA (CRS)	Success	Success (ground pad)
	Last launch of phase 1 of the CRS contract. Carries *Bartolomeo*, an ESA platform for hosting external payloads onto ISS.[508] Originally scheduled to launch on 2 March 2020, the launch date was pushed back due to a second stage engine failure. SpaceX decided to swap out the second stage instead of replacing the faulty part.[509] It was SpaceX's 50th successful landing of a first stage booster, the third flight of the Dragon C112 and the last launch of the cargo Dragon spacecraft.								
83	18 March 2020, 12:16[510]	F9 B5 △ B1048.5	KSC, LC-39A	Starlink 5 v1.0 (60 satellites)	15,600 kg (34,400 lb)[5]	LEO	SpaceX	Success	Failure (drone ship)
	Fifth operational launch of Starlink satellites. It was the first time a first stage booster flew for a fifth time and the second time the fairings were reused (Starlink flight in May 2019).[511] Towards the end of the first stage burn, the booster suffered premature shut down of an engine, the first of a Merlin 1D variant and first since the CRS-1 mission in October 2012. However, the payload still reached the targeted orbit.[512] This was the second Starlink launch booster landing failure in a row, later revealed to be caused by residual cleaning fluid trapped inside a sensor.[513]								
84	22 April 2020, 19:30[514]	F9 B5 △ B1051.4	KSC, LC-39A	Starlink 6 v1.0 (60 satellites)	15,600 kg (34,400 lb)[5]	LEO	SpaceX	Success	Success (drone ship)

Objectives

Web scrap Falcon 9 launch records with `BeautifulSoup`:

- Extract a Falcon 9 launch records HTML table from Wikipedia
- Parse the table and convert it into a Pandas data frame

First let's import required packages for this lab

In [15]:
```
!pip install beautifulsoup4
!pip install requests
!pip install pandas
```

```
Requirement already satisfied: beautifulsoup4 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (4.11.
1)
Requirement already satisfied: soupsieve>1.2 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from b
eautifulsoup4) (2.3.2.post1)
Requirement already satisfied: requests in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (2.29.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packa
ges (from requests) (3.1.0)
Requirement already satisfied: idna<4,>=2.5 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from re
quests) (3.4)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages
(from requests) (1.26.15)
Requirement already satisfied: certifi>=2017.4.17 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (f
rom requests) (2023.5.7)
Requirement already satisfied: pandas in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (1.3.5)
Requirement already satisfied: python-dateutil>=2.7.3 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-package
s (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2017.3 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from pa
ndas) (2023.3)
Requirement already satisfied: numpy>=1.17.3 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from p
andas) (1.21.6)
Requirement already satisfied: six>=1.5 in /home/jupyterlab/conda/envs/python/lib/python3.7/site-packages (from python
-dateutil>=2.7.3->pandas) (1.16.0)
```

In [16]:
```python
import sys

import requests
from bs4 import BeautifulSoup
import re
import unicodedata
import pandas as pd
```

and we will provide some helper functions for you to process web scraped HTML table

In [17]:
```python
def clean_text(cell):
    """Return the visible text of a table cell as one clean string.
    Reference markers (<sup>), line breaks and odd whitespace are removed.
    Works on a copy, so the original soup is NOT modified."""
    cell = BeautifulSoup(str(cell), "html.parser")
    for tag in cell.find_all(["sup", "style", "script"]):
        tag.decompose()
    for br in cell.find_all("br"):
```

```
        br.replace_with(" ")
    text = unicodedata.normalize("NFKD", cell.get_text(" "))
    text = re.sub(r"\s+", " ", text).strip()
    text = re.sub(r"\(\s+", "(", text)      # "( ISS )" -> "(ISS)"
    text = re.sub(r"\s+\)", ")", text)
    text = re.sub(r"\s+,", ",", text)
    return text
def extract_column_from_header(th):
    """Return the column name stored in a <th> header element (or None if empty)."""
    return clean_text(th) or None
def date_time(cell):
    """Return (date, time) strings from the 'Date and time (UTC)' cell."""
    cell = BeautifulSoup(str(cell), "html.parser")
    for tag in cell.find_all("sup"):
        tag.decompose()
    parts = [p.strip(" ,") for p in cell.stripped_strings]
    date = parts[0] if len(parts) > 0 else None
    time = parts[1] if len(parts) > 1 else None
    return date, time
def booster_version(cell):
    """Return the version + booster, e.g. 'F9 v1.0 B0003.1' (reference markers removed)."""
    cell = BeautifulSoup(str(cell), "html.parser")
    for tag in cell.find_all("sup"):
        tag.decompose()
    return " ".join(cell.stripped_strings) or None
def get_mass(cell):
    """Return the payload mass as text such as '4,700 kg', or None if the cell has no mass."""
    match = re.search(r"[\d,\.]+\s*kg", clean_text(cell))
    return match.group(0) if match else None
def landing_status(cell):
    """Return the booster landing result, e.g. 'Success', 'Failure (parachute)', 'No attempt'."""
    return clean_text(cell) or None
def launch_outcome(cell):
    """Return the launch outcome, e.g. 'Success' / 'Failure'."""
    text = clean_text(cell)
    return text.split(" ")[0] if text else None
def first_link_or_text(cell):
    """Return the text of the first link in the cell, falling back to the plain text."""
    link = cell.find("a")
    return link.get_text(strip=True) if link else (clean_text(cell) or None)
```

To keep the lab tasks consistent, you will be asked to scrape the data from a snapshot of the `List of Falcon 9 and Falcon Heavy launches` Wikipage updated on `9th June 2021`

In [19]:
```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches&oldid=1027686922"
# Wikipedia may reject requests without a browser-like User-Agent
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) "
                  "Chrome/91.0.4472.124 Safari/537.36"
}
```

Next, request the HTML page from the above URL and get a `response` object

TASK 1: Request the Falcon9 Launch Wiki page from its URL

First, let's perform an HTTP GET method to request the Falcon9 Launch HTML page, as an HTTP response.

In [20]:
```
response = requests.get(static_url, headers=headers, timeout=30)
response.raise_for_status()
# stop with a clear error if the request failed
print("Status code:", response.status_code)
```

```
Status code: 200
```

Create a `BeautifulSoup` object from the HTML `response`

In [21]:
```
# Use BeautifulSoup() on the response text content
soup = BeautifulSoup(response.text, "html.parser")
```

Print the page title to verify if the `BeautifulSoup` object was created properly

In [22]:
```
print("Page Title:", soup.title.string)
```

```
Page Title: List of Falcon 9 and Falcon Heavy launches - Wikipedia
```

TASK 2: Extract all column/variable names from the HTML table header

Next, we want to collect all relevant column names from the HTML table header

Let's try to find all tables on the wiki page first. If you need to refresh your memory about `BeautifulSoup`, please check the external reference link towards the end of this lab

In [23]:
```
html_tables = soup.find_all("table")
print(f"Found {len(html_tables)} tables on the page.")
```

```
Found 29 tables on the page.
```

Starting from the third table is our target table contains the actual launch records.

In [24]:
```
first_launch_table = html_tables[2]
print(first_launch_table.prettify()[:1500])
```

```
<table class="wikitable plainrowheaders collapsible" id="mwmQ" style="width: 100%;">
 <tbody id="mwmg">
  <tr id="mwmw">
   <th id="mwnA" scope="col">
    Flight No.
   </th>
   <th id="mwnQ" scope="col">
    Date and
    <br id="mwng"/>
    time (
    <a href="https://en.wikipedia.org/wiki/Coordinated_Universal_Time" id="mwnw" rel="mw:WikiLink" title="Coordinated
Universal Time">
     UTC
    </a>
    )
   </th>
   <th id="mwoA" scope="col">
    <a href="https://en.wikipedia.org/wiki/List_of_Falcon_9_first-stage_boosters" id="mwoQ" rel="mw:WikiLink" title="L
ist of Falcon 9 first-stage boosters">
     Version,
     <br id="mwog"/>
     Booster
    </a>
    <sup about="#mwt70" class="mw-ref reference" data-mw='{"name":"ref","attrs":{"name":"booster","group":"lower-alph
a"},"body":{"id":"mw-reference-text-cite_note-booster-11"},"parts":[{"template":{"target":{"wt":"efn","href":"./Templa
te:Efn"},"params":{"name":{"wt":"booster"},"1":{"wt":"Falcon 9 first-stage boosters are designated with a construction
serial number and an optional flight number when reused, e.g. B1021.1 and B1021.2 represent the two flights of booster
[[B1021]]. Launches using reused boosters are denoted with a recycled symbol ♺."}},"i":0}]}' id="cite_ref-booster_11
-0" rel="dc:references" typeof="mw:Transclusion mw:Extension/ref">
     <a data-mw-group="lower-alpha" href="#cite_note-booster-11" id="mwow">
      <span class="mw-reflink-text" id="mwpA">
       <span class="cite-bracket" id="mwpQ">
        [
```

You should able to see the columns names embedded in the table header elements `<th>` as follows:

```
<tr>
<th scope="col">Flight No.
</th>
```

```
<th scope="col">Date and<br/>time (<a href="/wiki/Coordinated_Universal_Time" title="Coordinated
Universal Time">UTC</a>)
</th>
<th scope="col"><a href="/wiki/List_of_Falcon_9_first-stage_boosters" title="List of Falcon 9
first-stage boosters">Version,<br/>Booster</a> <sup class="reference" id="cite_ref-booster_11-0">
<a href="#cite_note-booster-11">[b]</a></sup>
</th>
<th scope="col">Launch site
</th>
<th scope="col">Payload<sup class="reference" id="cite_ref-Dragon_12-0"><a href="#cite_note-
Dragon-12">[c]</a></sup>
</th>
<th scope="col">Payload mass
</th>
<th scope="col">Orbit
</th>
<th scope="col">Customer
</th>
<th scope="col">Launch<br/>outcome
</th>
<th scope="col"><a href="/wiki/Falcon_9_first-stage_landing_tests" title="Falcon 9 first-stage
landing tests">Booster<br/>landing</a>
</th></tr>
```

Next, we just need to iterate through the `<th>` elements and apply the provided `extract_column_from_header()` to extract column name one by one

In [25]:
```
column_names = []
# Only the first row (<tr>) of the table is the header; later <th> cells hold flight numbers
header_row = first_launch_table.find("tr")
for th in header_row.find_all("th"):
    name = extract_column_from_header(th)
    if name is not None and len(name) > 0:
        column_names.append(name)
print(column_names)
assert len(column_names) == 10, "Expected 10 header columns"
```

```
['Flight No.', 'Date and time (UTC)', 'Version, Booster', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Custome
r', 'Launch outcome', 'Booster landing']
```

Check the extracted column names

In [26]:
```
print(column_names)
```

```
['Flight No.', 'Date and time (UTC)', 'Version, Booster', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Custome
r', 'Launch outcome', 'Booster landing']
```

TASK 3: Create a data frame by parsing the launch HTML tables

We will create an empty dictionary with keys from the extracted column names in the previous task. Later, this dictionary will be converted into a Pandas dataframe

In [27]:
```
launch_dict = {
    "Flight No.": [],
    "Date": [],
    "Time": [],
    "Version Booster": [],
    "Launch site": [],
    "Payload": [],
    "Payload mass": [],
    "Orbit": [],
    "Customer": [],
    "Launch outcome": [],
    "Booster landing": [],
}
```

Next, we just need to fill up the `launch_dict` with launch records extracted from table rows.

Usually, HTML tables in Wiki pages are likely to contain unexpected annotations and other types of noises, such as reference links `B0004.1[8]`, missing values `N/A [e]`, inconsistent formatting, etc.

To simplify the parsing process, we have provided an incomplete code snippet below to help you to fill up the `launch_dict`. Please complete the following code snippet with TODOs or you can choose to write your own logic to parse all launch tables:

In [28]:
```
extracted_row = 0
# Every launch-year table on the page shares the same CSS classes
for table in soup.find_all("table", "wikitable plainrowheaders collapsible"):
```

```
    for row in table.find_all("tr"):
        # A launch row starts with a <th> holding the flight number; header and note rows do not
        th = row.find("th")
        if th is None:
            continue
        flight_number = clean_text(th)
        if not flight_number.isdigit():
            continue
        cells = row.find_all("td")
        if len(cells) < 9:          # skip incomplete/continuation rows (e.g. extra boosters)
            continue
        extracted_row += 1
        date, time = date_time(cells[0])
        launch_dict["Flight No."].append(flight_number)
        launch_dict["Date"].append(date)
        launch_dict["Time"].append(time)
        launch_dict["Version Booster"].append(booster_version(cells[1]))
        launch_dict["Launch site"].append(first_link_or_text(cells[2]))
        launch_dict["Payload"].append(clean_text(cells[3]))
        launch_dict["Payload mass"].append(get_mass(cells[4]))
        launch_dict["Orbit"].append(clean_text(cells[5]) or None)
        launch_dict["Customer"].append(clean_text(cells[6]) or None)
        launch_dict["Launch outcome"].append(launch_outcome(cells[7]))
        launch_dict["Booster landing"].append(landing_status(cells[8]))
print("Launch rows extracted:", extracted_row)
```

```
Launch rows extracted: 121
```

After you have fill in the parsed launch record values into `launch_dict`, you can create a dataframe from it.

In [29]:
```
df = pd.DataFrame(launch_dict)
print(df.shape)
df.head()
```

```
(121, 11)
```

Out[29]:

	Flight No.	Date	Time	Version Booster	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Booster landing
0	1	4 June 2010	18:45	F9 v1.0 B0003.1	CCAFS	Dragon Spacecraft Qualification Unit	None	LEO	SpaceX	Success	Failure (parachute)
1	2	8 December 2010	15:43	F9 v1.0 B0004.1	CCAFS	Dragon demo flight C1 (Dragon C101)	None	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2	3	22 May 2012	07:44	F9 v1.0 B0005.1	CCAFS	Dragon demo flight C2+ (Dragon C102)	525 kg	LEO (ISS)	NASA (COTS)	Success	No attempt
3	4	8 October 2012	00:35	F9 v1.0 B0006.1	CCAFS	SpaceX CRS-1 (Dragon C103)	4,700 kg	LEO (ISS)	NASA (CRS)	Success	No attempt
4	5	1 March 2013	15:10	F9 v1.0 B0007.1	CCAFS	SpaceX CRS-2 (Dragon C104)	4,877 kg	LEO (ISS)	NASA (CRS)	Success	No attempt

In [30]:
```
assert len(df) == extracted_row
print(df.isna().sum())
df.tail()
```

```
Flight No.         0
Date               0
Time               0
Version Booster    0
Launch site        0
Payload            0
Payload mass       5
Orbit              0
Customer           0
Launch outcome     0
Booster landing    0
dtype: int64
```

Out[30]:

	Flight No.	Date	Time	Version Booster	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Booster landing
116	117	9 May 2021	06:42	F9 B5 ♺ B1051.10	CCSFS	Starlink 27 v1.0 (60 satellites)	15,600 kg	LEO	SpaceX	Success	Success (drone ship)
117	118	15 May 2021	22:56	F9 B5 ♺ B1058.8	KSC	Starlink 26 v1.0 (52 Satellites) Capella -6 & ...	14,000 kg	LEO	SpaceX Capella Space and Tyvak	Success	Success (drone ship)
118	119	26 May 2021	18:59	F9 B5 ♺ B1063.2	CCSFS	Starlink 28 v1.0 (60 Satellites)	15,600 kg	LEO	SpaceX	Success	Success (drone ship)
119	120	3 June 2021	17:29	F9 B5 B1067.1	KSC	SpaceX CRS-22 (Dragon C209.1)	3,328 kg	LEO (ISS)	NASA (CRS)	Success	Success (drone ship)
120	121	6 June 2021	04:26	F9 B5 ♺ B1061.3	CCSFS	SXM-8	7,000 kg	GTO	Sirius XM	Success	Success (drone ship)

We can now export it to a **CSV** for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.

Following labs will be using a provided dataset to make each lab independent.

In [32]:
```
df.to_csv("spacex_web_scraped.csv", index=False)
```

Authors

Yan Luo

Nayef Abou Tayoun

Copyright © 2021 IBM Corporation. All rights reserved.